

Learning-Aided Safe Controller Synthesis with Formal Guarantees via Vector Barrier Certificates

1st Xia Zeng

Southwest University
xzeng0712@swu.edu.cn

2nd Mengxin Ren

East China Normal University
mxren@stu.ecnu.edu.cn

3rd Zhiming Liu

Southwest University
zhimingliu88@swu.edu.cn

4th Zhengfeng Yang*

East China Normal University
zfyang@sei.ecnu.edu.cn

Abstract—The design of controllers for safety-critical systems is an important research issue. Especially, the generation of controllers with formal safety guarantees is a challenging problem. Recently, for safety objectives of various system control tasks, machine learning technologies have been used to achieve ideal training and simulation performance, but formal guarantees are still lacking. This paper takes advantages of learning technology to assist safe controller synthesis with formal guarantees. On the one hand, the generation of verifiable safe controllers is aided by reinforcement learning; on the other hand, a set of barrier certificates (BC), i.e. a vector BC, is synthesized with the aid of deep learning to certify the safety of synthesized controllers. Vector BCs are more expressive than the conventional single BCs for safety verification. Compared with the existing work on vector BC generation, our method has two advantages: first, our method verifies a learned candidate vector BC, rather than directly generating a verified one, and thus has low computational complexity; second, the existing method has made relaxations to the non-convex vector BC constraints, which reduced the feasible region of solutions, while our method can deal with the original constraints. Furthermore, experiments fully demonstrate the effectiveness of our method on a series of benchmarks.

Index Terms—controller synthesis, safety verification, vector barrier certificate, reinforcement learning, sum-of-squares

I. INTRODUCTION

The design and synthesis of controllers are fundamental challenges in control theory. With the rapid advancement of deep learning, there has been a surge in research exploring the application of neural networks (NNs) for controlling nonlinear systems [1], [2]. NNs possess a robust capacity to represent complex nonlinear mappings and offer efficient computational performance, making them well-suited for sophisticated control tasks [3]. However, learned control policies often lack formal guarantees, limiting their applicability in practical scenarios, especially within safety-critical control systems. To ensure system safety under a designed controller over an indefinite time horizon, a crucial verification method is the generation of barrier certificates, which are employed to demonstrate that the trajectories of an autonomous dynamical system remain within a specified safe region. Since barrier

certificates provide sufficient conditions for safety assurance, researchers are continually exploring various forms of these functions to expand the range of verifiable problems.

The foundational work by Prajna et al. introduced the use of barrier certificates for the safety verification of hybrid systems [4]. Building upon this, subsequent research has focused on exploring various forms of barrier certificates to enrich and enhance their expressiveness. Kong et al. proposed barrier certificates based on exponential conditions for semi-algebraic systems [5]. Dai et al. discussed general relaxations of barrier certificate conditions that maintain convexity, broadening the scope of systems amenable to safety verification [6]. Sloth et al. introduced barrier certificates tailored for hybrid systems comprising multiple interconnected subsystems, facilitating compositional safety analysis [7]. Additionally, Zeng et al. explored Darboux-type barrier certificates, which define algebraic curves that system trajectories cannot cross once they intersect [8].

In fact, previous mainstream research on barrier certificates has predominantly utilized scalar functions. Recently, Sogokon et al. introduced a composite barrier function condition termed the vector barrier certificate (vector BC) [9]. When the combination number is set to one, it simplifies to a scalar barrier certificate, indicating that vector BCs offer broader expressiveness compared to conventional scalar barrier certificates. However, the theoretical advantages of vector BCs are accompanied by practical challenges. The existence conditions for vector BCs involve non-convex constraints, complicating the parameter-solving process. To facilitate computation, existing approaches have modified these non-convex constraints by strengthening the disjunctive conditions of multiple barrier certificates to one of the disjuncts. This modification, while simplifying the solving process, may result in the loss of feasible solutions to the original vector BC conditions. Besides, the so-called essentially non-negative matrix in the vector BC conditions needs to be handcrafted, which complicates its application.

In this paper, we focus on designing controllers with formal safety guarantees. We propose a novel method that leverages learning techniques to synthesize safe controllers for nonlinear systems using vector barrier certificates. Initially, we employ safe reinforcement learning combined with Probably Approximately Correct (PAC) approximation to obtain an ideal and verifiable safe controller candidate. Subsequently, we develop a new safety verification approach that utilizes

* Corresponding author

This work was supported in part by the National Key Research and Development Project, China under Grant 2022YFA1005101, the National Natural Science Foundation of China under Grants (No.62472362, 62272397, 12171159, 62032019), “Digital Silk Road” Shanghai International Joint Lab of Trustworthy Intelligent Software under Grant No. 22510750100, and Shanghai Trusted Industry Internet Software Collaborative Innovation Center.

learning techniques to assist in generating vector BCs. Unlike existing methods, our approach does not require modifying or strengthening the disjunctive constraints in the original vector BC conditions, and can learn the essentially non-negative matrix automatically, thereby avoiding solution loss and enhancing expressiveness and flexibility. In summary, this paper makes the following contributions: We leverage advanced learning techniques to propose a novel method for synthesizing safe controllers with formal guarantees; We have developed a novel approach that employs learning techniques to assist in the generation of vector BCs, through learning we avoid the drawbacks of conventional vector BC generation methods that strengthen constraints, causing the loss of potential feasible solutions for vector BCs; We propose an incremental SOS-encoding method to transform the formal verification of vector BC conditions into a series of semi-definite programs, enabling highly efficient posterior verification after learning; We conduct extensive comparative experiments on state-of-the-art benchmarks to demonstrate the effectiveness and robustness of our proposed method.

II. PRELIMINARIES

Notations. Let $\mathbb{R}$ be the field of real numbers, $\mathbb{R}[\mathbf{x}]$ be the ring of polynomials with coefficients in $\mathbb{R}$ over n -dimensional variables $\mathbf{x} = (x_1, x_1, \dots, x_n)^\top$, $\mathbb{R}[\mathbf{x}]^n$ be the space of n -dimensional vector polynomials, $\Sigma[\mathbf{x}] \subset \mathbb{R}[\mathbf{x}]$ be the space of Sum-of-Squares (SOS) polynomials, and $A_{i:}$ be the elements in the i -th row of a matrix A .

A. Safe Control Problem

A controlled continuous dynamical system is modeled by first-order ordinary differential equations

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) \\ \mathbf{u} = \pi(\mathbf{x}) \end{cases}, \quad (1)$$

where $\mathbf{x} \in D \subseteq \mathbb{R}^n$ is the system state, $\mathbf{u} \in U \subseteq \mathbb{R}^m$ is the control input, and $\mathbf{f} \in \mathbb{R}[\mathbf{x}]^n$ is the vector field defined on the state space $D \subseteq \mathbb{R}^n$. Assume $\mathbf{f}$ satisfies the local Lipschitz condition, which ensures (1) has a unique solution $\mathbf{x}(t, \mathbf{x}_0)$ in D for every initial state $\mathbf{x}_0 \in D$ at $t = 0$.

In many contexts, a dynamical system is equipped with a domain $\Psi \subset D$ and an initial set $\Theta \subset \Psi$, represented as a triple $\mathcal{C} \doteq (\mathbf{f}, \Psi, \Theta)$. Given a prespecified unsafe region $X_u \subset D$, we say that the system $\mathcal{C}$ is *safe* if all system trajectories starting from Θ can not evolve into any state specified by X_u , which has been widely investigated in safety critical applications.

Definition 1 (Safety): For a controlled constrained continuous dynamical system (CCDS) $\mathcal{C} = (\mathbf{f}, \Psi, \Theta)$ and a given unsafe region X_u , the system is safe if for all $\mathbf{x}_0 \in \Theta$, there does not exist $t_1 > 0$ s.t. $\mathbf{x}(t_1, \mathbf{x}_0) \in X_u \wedge \forall t \in [0, t_1]. \mathbf{x}(t, \mathbf{x}_0) \in \Psi$.

Definition 2 (Safe Controller Synthesis): Given a controlled CCDS $\mathcal{C} = (\mathbf{f}, \Psi, \Theta)$ with $\mathbf{f}$ defined by (1) with an unsafe set X_u , design a locally Lipschitz continuous feedback control law π such that the closed-loop system $\mathcal{C}$ with $\mathbf{f} = \mathbf{f}(\mathbf{x}, \pi(\mathbf{x}))$ is safe as per Definition 1.

In this paper, we consider the above safe controller synthesis problem for nonlinear systems with formal guarantee.

B. Vector Barrier Certificates

The concept of *barrier certificates* (BCs) plays an important role in safety verification of continuous systems. The essential idea is to use the zero level set of a function $B(\mathbf{x})$ as a barrier to separate all the reachable states from the unsafe region. Since the existence of BCs serves as a sufficient condition for system safety, efforts have been made to explore certificate functions that meet less rigid criteria. The concept of vector BCs is proposed in [9]. Compared to traditional scalar BCs, it represents a more expressive barrier certificate family.

Definition 3 (Vector Barrier Certificate): Let the system (1) and sets Ψ, Θ, X_u be given. An m -dimensional vector of continuously differentiable function $\mathbf{B}(\mathbf{x}) = (B_1(\mathbf{x}), B_2(\mathbf{x}), \dots, B_m(\mathbf{x}))^\top : \Psi \rightarrow \mathbb{R}^m$ is called a vector barrier certificate if it satisfies

- VBC1. $\forall \mathbf{x} \in \mathbb{R}^n. (\mathbf{x} \in \Theta \Rightarrow \bigwedge_{i=1}^m B_i(\mathbf{x}) \leq 0),$
- VBC2. $\forall \mathbf{x} \in \mathbb{R}^n. (\mathbf{x} \in X_u \Rightarrow \bigvee_{i=1}^m B_i(\mathbf{x}) > 0),$
- VBC3. $\forall \mathbf{x} \in \mathbb{R}^n. (\mathbf{x} \in \Psi \Rightarrow \mathcal{L}_{\mathbf{f}}\mathbf{B}(\mathbf{x}) \leq A\mathbf{B}(\mathbf{x})),$

where $\mathcal{L}_{\mathbf{f}}\mathbf{B}(\mathbf{x})$ denotes the element-wise Lie-derivative of $\mathbf{B}(\mathbf{x})$ along the vector field $\mathbf{f}(\mathbf{x})$, i.e., $\mathcal{L}_{\mathbf{f}}\mathbf{B}(\mathbf{x}) = (\mathcal{L}_{\mathbf{f}}B_1(\mathbf{x}), \mathcal{L}_{\mathbf{f}}B_2(\mathbf{x}), \dots, \mathcal{L}_{\mathbf{f}}B_m(\mathbf{x}))^\top$, and $A \in \mathbb{R}^{m \times m}$ is some *essentially non-negative* matrix.

Remark 1: When $m = 1$, the vector BC in Definition 3 reduces to a scalar function, which corresponds to the traditional single BC condition.

Definition 3 is developed by applying the comparison principle in the theory of ODEs to safety verification. An important prerequisite for this theorem is that the $m \times m$ real matrix A is essentially non-negative, i.e., all the off-diagonal entries of A are non-negative [10].

The following result from [9] can be used to guarantee the safety of a given controlled CCDS.

Theorem 1: Given a controlled CCDS $\mathcal{C} = (\mathbf{f}, \Psi, \Theta)$, with $\mathbf{f}$ defined by (1), a feedback control law $\mathbf{u} = \pi(\mathbf{x})$, and an unsafe region X_u , if an m -dimensional vector of continuously differentiable function $\mathbf{B}(\mathbf{x})$ as per Definition 3 exists, then the safety property of the system (1) is guaranteed.

Throughout this paper, we assume that the domain Ψ , the initial set Θ , and the unsafe set X_u are compact semi-algebraic sets, represented by polynomial equations and inequalities, i.e.,

$$\begin{cases} \Psi := \{\mathbf{x} \in \mathbb{R}^n \mid h_i(\mathbf{x}) \geq 0, 1 \leq i \leq m_1\}, \\ \Theta := \{\mathbf{x} \in \mathbb{R}^n \mid g_j(\mathbf{x}) \geq 0, 1 \leq j \leq m_2\}, \\ X_u := \{\mathbf{x} \in \mathbb{R}^n \mid q_k(\mathbf{x}) \geq 0, 1 \leq k \leq m_3\}, \end{cases} \quad (2)$$

for some polynomials $g_i, h_j, q_k \in \mathbb{R}[\mathbf{x}]$.

III. VERIFIABLE SAFE CONTROLLER SYNTHESIS VIA REINFORCEMENT LEARNING

In this section, we propose to synthesize an easily verifiable control policy by combining reinforcement learning (RL) and scenario optimization. RL is a promising tool for designing controllers, especially for complex systems. However, the intricate structure of deep neural network (DNN)

controllers trained through RL often makes their verification over unbounded time horizons challenging, limiting their use in safety-critical applications. To address this issue, we leverage well-performing learned controllers as a guide to design polynomial-type controllers based on Probably Approximately Correct (PAC) principles. Our approach probabilistically characterizes approximation errors and guides the design of polynomial templates for controlled approximation performance. Additionally, the polynomial forms of these controllers enable numerical methods for solving polynomial constraints, facilitating subsequent safety verification of the control system.

A. Training well-performing DNN controllers using RL

Here, we focus on how to train a well-performing controller by RL. Here *well-performing* means that the controller could lead the system to avoid the obstacles within the simulation time bound. Training with RL requires the construction of four components: the state space, the action space, the environment, and the reward function, respectively. For a control task, we take the system state $\mathbf{x}$ which is a vector of n dimensions as the state, the output of the controller u which is of dimension m as the action, and the system dynamics as the environment. Furthermore, the design of the reward function for the safety requirement is explained as follows.

We hope that RL helps to synthesize an ideal controller using the designed reward, and all the trajectories of the closed-loop system starting from the initial set Θ cannot evolve into the unsafe region X_u under the trained DNN controller. Thus, the reward function is preliminarily defined as $\hat{r}_t = \beta_1 \cdot \text{dist}(X_u, \mathbf{x}_t)$, where $\beta_1 > 0$ is the scale factor, and $\text{dist}(X_u, \mathbf{x}_t)$ denotes the distance between the state $\mathbf{x}_t$ and the unsafe region X_u .

In addition, once the trajectory approaches and nearly hits the boundary of the unsafe set, the system behavior should be penalized. For this purpose, the reward function is updated as

$$r_t = \begin{cases} \hat{r}(\mathbf{x}_t) - \min(\beta_2 \cdot \frac{1}{\text{dist}(X_u, \mathbf{x}_t)}, \Delta r_{\min}), & \mathbf{x} \in S_{belt} \\ \hat{r}(\mathbf{x}_t), & \text{otherwise} \end{cases}$$

where $S_{belt} = \{\mathbf{x} | \text{dist}(X_u, \mathbf{x}) < \delta\}$ with δ a small positive value, $\beta_2 > 0$ is the scale factor and $\Delta r_{\min} > 0$ is the threshold avoiding too large fluctuations of reward value. In this work, we set $\beta_1 = 1.0$, $\beta_2 = 5.0$, $\delta = 0.1$. The r_t set in this way can be kept within a certain range, making the convergence effect better.

The remaining problem is to train the controller via RL. Here we use Deep Deterministic Policy Gradient (DDPG) [1] which is a popular RL approach suited for continuous control applications. The DDPG algorithm combines the value-based and policy-based methods, and is made up of two neural networks: the critic network and actor network. Concretely, the critic network $Q : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$ takes the state $\mathbf{x}_t$ and the action $\mathbf{u}_t$ as input, and outputs a scalar Q-value $Q(\mathbf{x}_t, \mathbf{u}_t | \theta^Q)$, which evaluates the current action at the given state $\mathbf{x}_t$. The parameter θ^Q is updated by minimizing L^Q as

$$L^Q = \frac{1}{N} \sum_t (y_t - Q(\mathbf{x}_t, \mathbf{u}_t | \theta^Q))^2, \quad (3)$$

where $y_t = r_t + \gamma Q'(\mathbf{x}_{t+1}, \mathbf{u}'_{RL}(\mathbf{x}_{t+1}))$. The actor network $\mathbf{u}_{RL} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ receives a state $\mathbf{x}_t$ at time step t , and directly outputs a continuous action $\mathbf{u}_t = \mathbf{u}_{RL}(\mathbf{x}_t | \theta^A)$. The parameter θ^A is updated by minimizing J as

$$J = -\frac{1}{N} \sum_t Q(\mathbf{x}_t, \mathbf{u}_{RL}(\mathbf{x}_t | \theta^A)). \quad (4)$$

Note that in (3) and (4) the Q' and $\mathbf{u}'_{RL}$ are target networks that have the same structures as Q and $\mathbf{u}_{RL}$ respectively; in DDPG, the parameters of Q' and $\mathbf{u}'_{RL}$ are not learned but slowly synchronized with Q and $\mathbf{u}_{RL}$; besides, N is the training batch size, and $\gamma \in (0, 1)$ is the reward decay factor.

To train the desired controller, we first generate a set of initial states from Θ . For each sampled initial state $\mathbf{x}_0$, with the help of $\mathbf{u}_{RL}$, one may yield the associated trajectory as a discrete time state sequence $\{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_t, \dots, \mathbf{x}_T\}$ which does not enter the unsafe area, and then collect the transition tuples $(\mathbf{x}_t, \mathbf{x}_{t+1}, \mathbf{u}_t, r_t)$ to form a replay buffer. Every few time steps, a batch size of data is sampled from the replay buffer to update the parameters θ^Q and θ^A , and then the new controller is used to simulate the trajectory to collect new data until the controller behaves well.

B. Polynomial controller synthesis based on PAC-model

Based on the trained DNN controller $\mathbf{u}_{RL}(\mathbf{x})$ through RL, we construct an easily verifiable controller with a polynomial form based on Probably Approximately Correct (PAC) principles [11]. The error bound between the PAC-based polynomial controller and the original auxiliary controller $\mathbf{u}_{RL}(\mathbf{x})$ can be evaluated in a statistical way given a pre-defined error rate ϵ and a significance level η .

We first seek to compute an approximate polynomial $p(\mathbf{x}, \mathbf{c}) \in \mathbb{R}[\mathbf{x}]$ with a pre-assigned degree d . Let $[\mathbf{x}]_d$ be the vector consisting of monomials whose degree is at most d , ordered in the graded lexicographic ordering, i.e.,

$$[\mathbf{x}]_d = (1, x_1, x_2, \dots, x_n, x_1^2, x_1 x_2, \dots, x_{n-1} x_n^{d-1}, x_n^d)^\top.$$

Let the dimension of $[\mathbf{x}]_d$ be ν , i.e., $\nu = \dim([\mathbf{x}]_d) = \binom{n+d}{d}$. Define $\mathbb{N}_d^n$ as $\{\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{N}^n \mid \sum_1^n \alpha_i \leq d\}$ and the monomial $\mathbf{x}^\alpha = x_1^{\alpha_1} x_2^{\alpha_2} \dots x_n^{\alpha_n}$. Thus, $p(\mathbf{x}, \mathbf{c})$ can be written as

$$p(\mathbf{x}, \mathbf{c}) = \mathbf{c}^T [\mathbf{x}]_d = \sum_{\alpha \in \mathbb{N}_d^n} \mathbf{c}_\alpha \mathbf{x}^\alpha.$$

Essentially, we aim to find a polynomial $p(\mathbf{x}, \mathbf{c})$ that approximates $\mathbf{u}_{RL}(\mathbf{x})$ as closely as possible, which amounts to solve

$$\min_{\mathbf{c}} e \quad s.t. \quad |\mathbf{u}_{RL}(\mathbf{x}) - p(\mathbf{x}, \mathbf{c})| \leq e, \quad \forall \mathbf{x} \in \Psi. \quad (5)$$

However, it is impossible for a polynomial of a specified degree to be an arbitrarily close approximation to a DNN within a specified region. In practice, we seek for a polynomial $p(\mathbf{x}, \mathbf{c})$ that approximates $\mathbf{u}_{RL}(\mathbf{x})$ to a high degree of accuracy, that is, with an approximation error less than a threshold τ .

Theoretically, solving (5) is a challenge due to the presence of infinitely many constraints. Calafiore et al. introduced a scenario approach to address this issue, providing a solution with a PAC guarantee in [12].

The scenario approach relaxes the infinitely many constraints in (5) by only considering a finite subset of K constraints. Meanwhile, a PAC guarantee depending on K , between the original solution in (5) is proved in [12] and the following scenario solution in (6).

For pre-specified ϵ and η , we seek a polynomial $p(\mathbf{x}, \mathbf{c})$ with degree d by solving the transformed linear programming from (5) using the scenario optimization approach, i.e., by solving:

$$\min_{\mathbf{c}} e \quad \text{s.t.} \quad |\mathbf{u}_{RL}(\mathbf{x}_i) - p(\mathbf{x}_i, \mathbf{c})| \leq e, \quad \forall \mathbf{x}_i \in \tilde{\Psi}, \quad (6)$$

where $\tilde{\Psi}$ is constructed by randomly sampling from Ψ . According to PAC principles [11], there are requirements for the number of sampling points, i.e., $K = |\tilde{\Psi}|$, to achieve credible approximation error $\epsilon \geq \frac{2}{K}(\ln \frac{1}{\eta} + \kappa)$ where $\kappa = \nu + 1$.

Therefore, through solving the linear programming (6), we obtain a polynomial $p(\mathbf{x}, \mathbf{c}^*)$ which is an approximation of $\mathbf{u}_{RL}(\mathbf{x})$ with a statistical error bound e^* at a given confidence level. Furthermore, if e^* exceeds the specified empirical threshold τ , the error bound between the approximate polynomial and the auxiliary controller $\mathbf{u}_{RL}(\mathbf{x})$ can be further improved by increasing the degree of the polynomial template and re-solving the problem (6), which increases the dimensionality of the feasible domain for the polynomial coefficients.

IV. LEARNING-AIDED SAFETY VERIFICATION WITH VECTOR BARRIER CERTIFICATES GENERATION

We have synthesized a polynomial controller for a controlled CCDS subject to the safety requirement with the aid of RL learning and PAC-model approximation. In this section, we propose to generate a vector BC as stated in Definition 3 to ensure the safety property in formal reasoning. For conventional scalar BC generation, the SOS relaxation technique is often used to effectively encode barrier certificate conditions, transforming them into solvable problems such as LMIs or SDPs. However, for the extended vector BC conditions, directly using SOS encoding causes the VBC2 condition to lead to non-convexity, making subsequent computations unsolvable. To address this difficulty, we design a learning-assisted method for vector BC generation, followed by SOS-based post-verification.

A. Vector Barrier Certificate Learning

The safety verification of polynomial controllers obtained through RL combined with PAC approximation is based on synthesis of the vector BC in Definition 3. Existing methods consider using SOS encoding of the conditions in Definition 3 to solve for the polynomial-form vector BC and verify the system's safety [9]. Due to the non-convexity of the VBC2 constraint in Definition 3, this method requires further strengthening and modifying the constraint as

$$\forall \mathbf{x} \in \mathbb{R}^n. (\mathbf{x} \in X_u \Rightarrow B_{i^*}(\mathbf{x}) > 0)$$

for some $i^* \in \{1, \dots, m\}$, to recover convexity. The introduction of such stronger conditions reduces the feasibility of the constraints with respect to the existence of vector BCs.

We leverage learning techniques to directly obtain vector BCs based on the original conditions and then design an SOS-based post-verification method tailored to disjunctive expressions, thereby avoiding the solution loss caused by strengthened conditions.

1) *vector BC network design*: To facilitate the use of numerical SOS verification methods in subsequent studies, we consider constructing a polynomial-form vector BC. A straightforward approach is to build a polynomial network by weighting neurons, followed by applying a commonly used *square* activation function. Instead of the polynomial network of with square activation functions, we make use of the DNN jump connection proposed in [13] to construct a Skip polynomial network.

Let $\mathbf{x}^{(0)} \in \mathbb{R}^n$ denote the input of the Square/Skip network, $W_1^{(k+1)}$ and $b_1^{(k+1)}$ be the weight matrix and the bias from the k -th layer to the $(k+1)$ -th layer respectively, and $W_2^{(k+1)}$ and $b_2^{(k+1)}$ be the weight matrix and the bias from the 0-th layer to the $(k+1)$ -th layer respectively. Square network multiplies the output of the current layer with itself as the input of the next layer, i.e., the recursive relationship two adjacent layers of the Square network can be formally expressed as:

$$\mathbf{x}^{(k+1)} = (W_1^{(k+1)} \mathbf{x}^{(k)} + b_1^{(k+1)})^2.$$

As can be seen from the module of the *skip polynomial network* in Fig. 1, the Skip network directly multiplies the output of the current layer with the input layer as the input for the next layer. Thus the relationship between two adjacent layers of the Skip network can be expressed as:

$$\mathbf{x}^{(k+1)} = (W_1^{(k+1)} \mathbf{x}^{(k)} + b_1^{(k+1)}) \otimes (W_2^{(k+1)} \mathbf{x}^{(0)} + b_2^{(k+1)}),$$

where $\otimes$ is the Hadamard product.

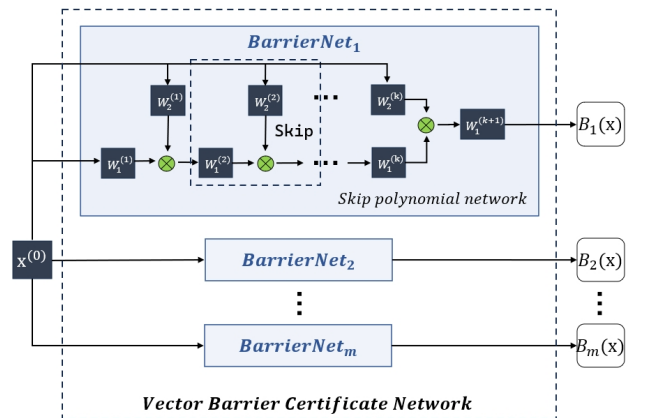


Fig. 1. Structure of vector BC Network.

Compared with Square network, the Skip network is more expressive by introducing two different weights and bias parameters on each layer. In this paper, we make use of Skip polynomial network as a basic module to construct a Skip

polynomial vector network (see Fig. 1) and Skip polynomial matrix network for vector BC and multiplier A respectively.

2) *loss function design*: To learn a vector BC that satisfies the three constraints in Definition 3, the designed vector BC network structure is utilized to learn $B(\mathbf{x}, \theta) = (B_1(\mathbf{x}, \theta_1), \dots, B_m(\mathbf{x}, \theta_m))^T$ and the auxiliary multiplier matrix $A(\Lambda)$, thereby obtaining parameters θ and Λ that satisfy the safety constraints. The vector BC parameters are learned by minimizing the loss function:

$$\mathcal{L}(\theta, \Lambda) = \mathcal{L}_{init}(\theta) + \mathcal{L}_{Unsafe}(\theta) + \mathcal{L}_d(\theta, \Lambda),$$

where $\mathcal{L}_{init}(\cdot)$, $\mathcal{L}_{Unsafe}(\cdot)$ and $\mathcal{L}_d(\cdot)$ are loss sub-functions designed to satisfy VBC1, VBC2, and VBC3, respectively.

The training dataset is constructed by randomly sampling from the system's initial set Θ , unsafe set X_u and system domain Ψ , denoted as $\hat{\Theta}$, $\hat{X}_u$ and $\hat{\Psi}$, respectively. For *condition VBC1*, it requires that $\forall \mathbf{x} \in \Theta, B_i(\mathbf{x}) \leq 0, 1 \leq i \leq m$. To encode this condition, we design the loss sub-function $\mathcal{L}_{init}$ as follows: $\sum_{x_i \in \hat{\Theta}} \text{Relu}(\max\{B_1(x_i), \dots, B_m(x_i)\})$. For *condition VBC2*, it requires that $\forall \mathbf{x} \in X_u, B_i(\mathbf{x}) > 0$ for some $i \in \{1, \dots, m\}$. Then, we design the loss sub-function $\mathcal{L}_{Unsafe}$ as follows: $\sum_{x_i \in \hat{X}_u} \text{Relu}(-\max\{B_1(x_i), \dots, B_m(x_i)\})$. For *condition VBC3*, it requires that $\mathcal{L}_f B(\mathbf{x}) \leq A B(\mathbf{x})$ on the system domain Ψ , i.e., $\mathcal{L}_f B_i(\mathbf{x}) - A_{i,:} B(\mathbf{x}) \leq 0$ for $\mathbf{x} \in \Psi$, and thus we design the loss sub-function $\mathcal{L}_d$ as: $\sum_{x_i \in \hat{\Psi}} \text{Relu}(\max_{1 \leq j \leq m} \{\mathcal{L}_f B_j(x_i) - A_{j,:} B(x_i)\})$. Then, the *AdamW optimizer* is applied to minimize $\mathcal{L}(\theta, \Lambda)$ subject to the constraints that all the off-diagonal entries of A are non-negative to generate a vector barrier function network candidate $B(\mathbf{x})$.

Example 1: Consider the following closed-loop system

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_1^2 + x_2^2 - 1 \\ 5(x_1 x_2 - 1) \end{bmatrix},$$

with $\Psi = \{x_1, x_2 \in \mathbb{R} \mid -4 \leq x_1 \leq 4, -4 \leq x_2 \leq 4\}$, $\Theta = \{x_1, x_2 \in \mathbb{R} \mid (x_1 - 0.6)^2 + (x_2 - 0.2)^2 \leq 0.5^2\}$, and $X_u = \{x_1, x_2 \in \mathbb{R} \mid -1.8 \leq x_1 \leq 2, 1 \leq x_2 \leq 2\}$, for which we learned a vector BC $(B_1, B_2)^T$ with $B_1(\mathbf{x}) = -1.284x_1 + 0.519x_2 - 0.665$ and $B_2(\mathbf{x}) = 0.149x_1 + 0.619x_2 - 0.542$. Please see Fig. 2 for an illustration.

B. Posterior Verification

In this subsection, we verify that the learned vector barrier certificate candidate $B(\mathbf{x}) = (B_1(\mathbf{x}), B_2(\mathbf{x}), \dots, B_m(\mathbf{x}))^T$ strictly satisfies the conditions VBC1, VBC2, and VBC3. This is achieved by transforming the verification problems into global optimization problems over polynomials, which can be solved efficiently via SOS-encoding and semidefinite program solving.

Given a compact basic semi-algebraic set $\mathbb{K}$ defined by $\mathbb{K} = \{\mathbf{x} \in \mathbb{R}^n \mid \gamma_1(\mathbf{x}) \geq 0, \dots, \gamma_\ell(\mathbf{x}) \geq 0\}$, where $\gamma_j(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$ for $1 \leq j \leq \ell$, the global optimization problem $\min_{\mathbf{x} \in \mathbb{K}} f(\mathbf{x})$ is well defined for any polynomial $f(\mathbf{x}) \in \mathbb{R}[\mathbf{x}]$. It is not difficult to check that

$$\forall \mathbf{x} \in \mathbb{R}^n. (\mathbf{x} \in \mathbb{K} \Rightarrow f(\mathbf{x}) \triangleright 0) \equiv \min_{\mathbf{x} \in \mathbb{K}} f(\mathbf{x}) \triangleright 0 \quad , \quad (7)$$

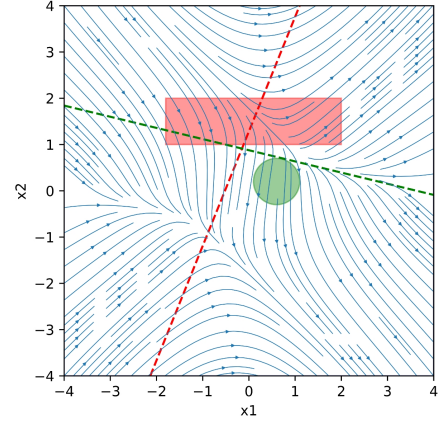


Fig. 2. An illustrative example with the learned vector barrier certificate $(B_1, B_2)^T$. The outer white square, the green circle, and the red rectangle represent Ψ , Θ and X_u , respectively. The red and green dotted lines represent the level sets of B_1 and B_2 , respectively, which are formally verified by SOS programming. The blue curves with arrows represent the system trajectories.

where $\triangleright \in \{>, \geq\}$, and $\equiv$ means the two constraints on both sides are equivalent. By [14], $\min_{\mathbf{x} \in \mathbb{K}} f(\mathbf{x})$ can be well approximated from below as closely as possible by solving a hierarchy of SOS-programs

$$\max_{r \in \mathbb{R}, \sigma_i \in \Sigma[\mathbf{x}]} r \quad \text{s.t.} \quad (f(\mathbf{x}) - r - \sum_{i=1}^{\ell} \sigma_i(\mathbf{x}) \gamma_i(\mathbf{x})) \text{ is SOS}, \quad (8)$$

with increasing degree bounds for σ_i .

Combining Definition 3 and formulas (2) and (7), we can see that VBC1 is equivalent to

$$\min_{\mathbf{x} \in \Theta} -B_i(\mathbf{x}) \geq 0, \quad 1 \leq i \leq m,$$

which can be checked by solving m optimizations as (8); similarly, VBC3 is equivalent to

$$\min_{\mathbf{x} \in \Psi} (A_{j,:} B(\mathbf{x}) - \mathcal{L}_f B_j(\mathbf{x})) \geq 0, \quad 1 \leq j \leq m,$$

which can again be checked via m optimizations as (8).

Verification of VBC2 is a bit more complicated, which will be conducted iteratively as the following four steps:

- Initially set $i = 1$ and $X = X_u$;
- Compute $r_i = \min_{\mathbf{x} \in X} B_i(\mathbf{x})$ by (8);
- If $r_i > 0$ then we are done; otherwise, update X by $X \wedge B_i(\mathbf{x}) \leq 0$ and increment i by 1;
- If $i \leq m$, go to the second step; otherwise we fail.

Therefore at most m optimizations as (8) need to be solved.

We have successfully applied the SOS-based approach to formally verify the learned vector BC of Example 1.

V. EXPERIMENTS

In this section, we conduct and report an experimental evaluation of our method over a set of benchmark examples. We further conduct experiments to compare our method with a mainstream SOS-based BC generation approach in terms of safety verification performance. All experiments are conducted

on a machine with Ubuntu 22.04, 256GB RAM, dual AMD EPYC 7302 CPUs, and three NVIDIA RTX A6000 GPUs.

The original conditions for the vector BCs we adopted were first proposed in [9]. Compared to [9], their method is not applicable to the safe controller generation problem. To facilitate computation, they strengthened the original vector BC conditions. Furthermore, when addressing the existence of vector BCs, their approach requires manually fixing a key multiplier matrix A in advance. However, as they discussed in their paper, there is no well-established guidance for A 's setting. Since the setting of A remains unclear, we do not conduct a detailed comparison with [9]. Moreover, they only reported a case study on one 3-dimensional verification problem.

We present a detailed experimental evaluation on a set of widely used benchmarks C_1 to C_{10} in Table I, using our proposed Learning-Aided Safe Controller Synthesis with vector BC method (*Lasac_VBC*). To further demonstrate the verification effectiveness of the learning-assisted vector BC generation in our *Lasac_VBC*, we compared it with the conventional approach of scalar BC generating using *SOSTOOLS* [15] for safety verification.

The origins of these 10 widely used cases are provided in the first column; n_x and d_f denote the number of state variables and the maximal degree of the polynomials (or the polynomial abstractions for non-polynomial systems) in the vector fields, and the examples are with dimension up to 12.

For each control task, *Lasac_VBC* first obtains a well-performing controller through RL and PAC approximation. The auxiliary network structure information and the corresponding polynomial degree of the controller are recorded in the 'NN struc.' and ' $d_{contr.}$ ' columns, respectively. Here, all our used NNs are with ReLU activation functions. Secondly, the results of the learned vector BC, including its dimension (combination parameter m) and the degree of each B_i , are recorded in the ' m_{vbc} ' and ' d_{vbc} ' columns, respectively. Finally, the learned vector BC is subjected to post-verification, and the verification time cost is recorded in the ' $T_{vbc}(s)$ ' column if successful. Meanwhile, in the comparison of safety verification performance, *SOSTOOLS* uses the safe controller learned by *Lasac_VBC* for each control case. *SOSTOOLS* starts the search with a low-degree polynomial template, incrementally increasing the polynomial degree until a barrier certificate is found. The resulting degree and computation time are recorded in the ' d_B ' and ' $T_B(s)$ ' columns, respectively. If the search fails to find a barrier certificate after attempting up to a degree 6 polynomial template, it is reported as a verification failure by '×'.

Observing Table I, our *Lasac_VBC* successfully synthesizes safe controllers with vector BC guarantees for all examples including a complex 12-dimensional system. In contrast, *SOSTOOLS* serves only as a safety verification method and fails to verify C_8 , C_9 and C_{10} . Comparing the safety verification performance: for examples C_1 - C_7 , the time efficiency of the two methods can be compared by observing the ' $T_{vbc}(s)$ ' and ' $T_B(s)$ ' columns. It is evident that for cases where *SOSTOOLS* can also perform verification, *Lasac_VBC*

TABLE I
PERFORMANCE EVALUATION OF OUR METHOD

Benchs.	n_x	d_f	<i>Lasac_VBC</i>					<i>SOSTOOLS</i>	
			NN struc.	$d_{contr.}$	m_{vbc}	d_{vbc}	$T_{vbc}(s)$	d_B	$T_B(s)$
C_1 [16]	2	2	2-64(2)-1	2	2	1,1	0.377	2	3.241
C_2 [16]	2	2	2-64(2)-1	2	2	1,1	0.343	2	3.997
C_3 [17]	3	3	3-64(2)-1	2	2	1,1	1.258	2	4.867
C_4 [18]	3	2	3-64(2)-1	2	2	1,1	0.766	2	4.652
C_5 [19]	4	3	4-64(2)-1	2	2	2,2	1.900	2	6.584
C_6 [20]	5	2	5-64(2)-1	2	2	1,1	3.574	2	8.810
C_7 [17]	6	3	6-64(2)-1	2	2	2,2	11.702	2	12.833
C_8 [21]	7	2	7-64(2)-1	2	2	1,1	10.626	×	×
C_9 [21]	9	2	9-64(2)-1	2	2	1,2	31.454	×	×
C_{10} [22]	12	1	12-64(2)-1	2	2	1,1	107.999	×	×

consistently achieves better time efficiency than *SOSTOOLS*.

Further examining the ' d_{vbc} ' and ' d_B ' columns in Table I, in all examples, the vector BCs successfully generated by *Lasac_VBC* are combinations of low-degree BCs, with their degrees not exceeding those obtained by *SOSTOOLS*. Additionally, in most examples, the degrees of our results are lower than those obtained by *SOSTOOLS*.

This indicates that the vector BCs generated by *Lasac_VBC*, through the vector form as a combination of multiple BCs, can achieve the same expressiveness that conventional single BCs require higher degrees to attain. The use of multiple low-degree functions for verification, compared to high-degree complex functions, offers potential advantages in terms of verification computational efficiency and the scale of problems that can be handled. The superior performance of *Lasac_VBC* over *SOSTOOLS* in terms of experimental verification efficiency and the scale of verification problems precisely demonstrates this point.

To summarize, Table I shows that our proposed *Lasac_VBC* demonstrates potential advantages in learning-assisted safe controller generation, verification computational efficiency, and the scale of problems it can handle.

VI. CONCLUSION

This paper addresses the problem of controller generation for safety-critical systems by skillfully leveraging RL, PAC techniques, and supervised learning to assist in the generation of formally guaranteed safe controllers. Benefiting from a controller synthesis method that combines RL and PAC approximation techniques, our approach yields system control policies with good behavioral performance. Additionally, learning techniques are utilized to aid in generating vector BCs with higher expressiveness, reducing verification costs while avoiding the drawbacks of conventional methods that strengthen constraints, causing the loss of potential feasible solutions for vector BCs. Especially, we propose an incremental SOS-encoding method to transform the formal verification of vector BC conditions into a series of semi-definite programs, enabling highly efficient solving. The experimental results demonstrate the advantages of the proposed method in learning-assisted controller synthesis and verification.

REFERENCES

- [1] T. P. Lillicrap, J. J. Hunt, A. Pritzel, and et al., “Continuous control with deep reinforcement learning,” *Computer science*, 2015.
- [2] Y. Duan, X. Chen, R. Houthoofd, J. Schulman, and P. Abbeel, “Benchmarking deep reinforcement learning for continuous control,” in *International conference on machine learning*. PMLR, 2016, pp. 1329–1338.
- [3] A. S. Poznyak, E. N. Sanchez, and W. Yu, *Differential neural networks for robust nonlinear control: identification, state estimation and trajectory tracking*. World Scientific, 2001.
- [4] S. Prajna and A. Jadbabaie, “Safety verification of hybrid systems using barrier certificates,” in *International Workshop on Hybrid Systems: Computation and Control*. Springer, 2004, pp. 477–492.
- [5] H. Kong, F. He, X. Song, W. N. Hung, and M. Gu, “Exponential-condition-based barrier certificate generation for safety verification of hybrid systems,” in *International Conference on Computer Aided Verification*. Springer, 2013, pp. 242–257.
- [6] L. Dai, T. Gan, B. Xia, and N. Zhan, “Barrier certificates revisited,” *Journal of Symbolic Computation*, vol. 80, pp. 62–86, 2017.
- [7] C. Sloth, G. J. Pappas, and R. Wisniewski, “Compositional safety analysis using barrier certificates,” in *Proceedings of the 15th ACM international conference on Hybrid Systems: Computation and Control*, 2012, pp. 15–24.
- [8] X. Zeng, W. Lin, Z. Yang, X. Chen, and L. Wang, “Darboux-type barrier certificates for safety verification of nonlinear hybrid systems,” in *Proceedings of the 13th International Conference on Embedded Software*, 2016, pp. 1–10.
- [9] A. Sogokon, K. Ghorbal, Y. K. Tan, and A. Platzer, “Vector barrier certificates and comparison systems,” in *Formal Methods - 22nd International Symposium, FM 2018*, ser. Lecture Notes in Computer Science, vol. 10951. Springer, 2018, pp. 418–437.
- [10] W. Walter, “Differential inequalities and maximum principles: theory, new methods, and applications,” *Nonlinear Analysis: Theory, Methods & Applications*, vol. 30, no. 9, pp. 4695–4711, 1997.
- [11] G. Calafiore and M. Campi, “The scenario approach to robust control design,” *IEEE Transactions on automatic control*, vol. 51, no. 5, pp. 742–753, 2006.
- [12] M. Campi, S. Garatti, and M. Prandini, “The scenario approach for systems and control design,” *Annual Reviews in Control*, vol. 33, no. 2, pp. 149–157, 2009.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2016, pp. 770–778.
- [14] J. B. Lasserre, *An Introduction to Polynomial and Semi-Algebraic Optimization*, ser. Cambridge Texts in Applied Mathematics. Cambridge University Press, 2015, vol. 52.
- [15] S. Prajna, A. Papachristodoulou, and P. A. Parrilo, “Sostools: Sum of squares optimization toolbox for matlab,” 2002. [Online]. Available: <https://api.semanticscholar.org/CorpusID:56539680>
- [16] A. Sogokon, K. Ghorbal, and T. T. Johnson, “Non-linear continuous systems for safety verification (benchmark proposal),” in *ARCH@ CPSWeek 2016-3rd International Workshop on Applied Verification for Continuous and Hybrid Systems*, vol. 43. EasyChair, 2016, pp. 42–51.
- [17] X. Chen, C. Peng, W. Lin, Z. Yang, Y. Zhang, and X. Li, “A novel approach for solving the bmi problem in barrier certificates generation,” in *International Conference on Computer Aided Verification*. Springer, 2020, pp. 582–603.
- [18] J. V. Deshmukh, J. P. Kapinski, T. Yamaguchi, and D. Prokhorov, “Learning deep neural network controllers for dynamical systems with safety guarantees,” in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–7.
- [19] G. Chesi, “Computing output feedback controllers to enlarge the domain of attraction in polynomial systems,” *IEEE Transactions on Automatic Control*, vol. 49, no. 10, pp. 1846–1853, 2004.
- [20] M. A. B. Sassi and S. Sankaranarayanan, “Stabilization of polynomial dynamical systems using linear programming based on bernstein polynomials,” *arXiv preprint arXiv:1501.04578*, 2015.
- [21] E. Klipp, R. Herwig, A. Kowald, C. Wierling, and H. Lehrach, *Systems biology in practice: concepts, implementation and application*. John Wiley & Sons, 2005.
- [22] S. Gao, “Quadcopter model,” [online] <https://github.com/dreal/benchmarks>, 2016.